Side-effects – Practice Class Work

Student: Ramazonov Norbek SE16054 Group: SE1-24 Date: 27.10.2026

Part A. Theory

Question 1

- **Definition of a Side-Effect:** An operation that changes the state of the app outside the scope of a Composable function (e.g., network requests, database writes, logging, analytics).
- **Why the Composable body is dangerous:** Composable functions can execute frequently, concurrently, or be discarded and restarted during recomposition. Placing side-effects directly in the body causes them to run multiple times unpredictably, leading to duplicate network requests or log entries.
- **Lecture Definition of an Effect:** A piece of code that runs outside the UI composition lifecycle or safely bridges Compose state with non-Compose code.
- **Why data loading belongs in the ViewModel:** ViewModels survive configuration changes and maintain state independently of UI recompositions. This preserves Unidirectional Data Flow (UDF), ensuring the UI remains a pure function of its state without triggering asynchronous side-effects during rendering.

Question 2

I want to	**API**
call a suspend function while the composable is in the Composition	LaunchedEffect
launch a coroutine from a callback such as onClick	rememberCoroutineScope
read the latest value inside an effect without restarting it	rememberUpdatedState
register a listener and remove it afterwards	DisposableEffect
publish Compose state to code Compose does not manage	SideEffect
turn Flow, LiveData or RxJava into Compose State	produceState
recompose less often than the input changes	derivedStateOf
turn Compose State into a Flow	snapshotFlow

Question 3

1. **When does the effect start?** As soon as the composable enters the Composition.
2. **What happens when pulseRateMs changes from 3000 to 1000?** The running coroutine is cancelled, and a new one starts immediately with the new delay value.
3. **Leaving Composition & isActive:** When the composable leaves the Composition, the coroutine scope is cancelled. isActive is a coroutine property checking if the coroutine is still active. The MiniComposition model shows effect(...) cancelled when disposed.

Question 4

- **Why suspicious?** LaunchedEffect(true) runs only once when entering composition and never restarts, ignoring changes to userId, exactly like an infinite while(true) loop.
- **When appropriate:** For one-time operations or splash screens where inputs never change.
- **Fixed Code:**

  Kotlin

  ```
  LaunchedEffect(userId) {
     api.sendHello(userId)
  }
  ```

Question 5

- **Why it fails in a callback:** LaunchedEffect is a Composable function and must be called directly within the Composition tree, not inside imperative event callbacks like onClick.
- **rememberCoroutineScope:** Returns a coroutine scope tied to the Composable's lifecycle, which is automatically cancelled when the composable leaves the Composition.
- **Correct version:**

  Kotlin

  ```
  val scope = rememberCoroutineScope()
  Button(onClick = { scope.launch { snackbarHostState.showSnackbar("Hi") } }) {
     Text("Show")
  }
  ```

- **Why showSnackbar is a suspend function:** Showing UI feedback requires asynchronous awaiting until the snackbar is dismissed or times out.
- **API selection:** Use LaunchedEffect for work that starts because the composable appeared; use rememberCoroutineScope for work triggered by user actions in callbacks.

Question 6

- **Stale lambda issue:** LaunchedEffect(true) never restarts, capturing the initial onTimeout lambda from the first composition, even if the parent passes a newer lambda later.
- **How rememberUpdatedState helps:** It stores the latest lambda reference without restarting the underlying effect coroutine.
- **Why making onTimeout a key is wrong:** A new lambda reference is created on every parent recomposition, which would constantly restart the timer.
- **Model output proof:** Proves that currentOnTimeout successfully invokes the latest version (v3) while keeping the coroutine running uninterrupted.

Question 7

- **Why cleanup is needed:** Unremoved observers cause memory leaks and multiple duplicate callbacks when recomposing or recreating the view.
- **onDispose rule:** Must appear as the final statement inside DisposableEffect. Otherwise, it causes a compilation error.
- **Empty onDispose:** Bad practice because it leaves cleanup incomplete. Better effects like DisposableEffect with proper cleanup or lifecycle hooks should be used.
- **Fixed Code:**

  Kotlin

```
DisposableEffect(lifecycleOwner) {
    val observer = LifecycleEventObserver { _, _ -> /* ... */ }
    lifecycleOwner.lifecycle.addObserver(observer)
    onDispose {
        lifecycleOwner.lifecycle.removeObserver(observer)
    }
}
```

Question 8

- **Console Output (four lines):**

  Plaintext

```
A.addObserver(observer)
A.removeObserver(observer)
B.addObserver(observer)
B.removeObserver(observer)
```

- **Why second screen("A") prints nothing:** The key (owner) did not change, so the effect did not restart.
- **Order when B arrives:** Removing the old observer (A) happens *before* adding the new one (B).

Question 9

- **When SideEffect runs:** After every successful recomposition.

- **Why direct call is dangerous:** It executes during the composition phase. If a composition attempt is discarded due to recomposition race conditions, the direct call still fires prematurely.
- **Model attempts behavior:** Direct calls fire even on discarded attempts (commits=false). SideEffect only executes when commits=true (successful commit).
- **Does it take keys?** No, because it runs on every successful commit.

Question 10

- **What produceState does:** Converts non-Compose asynchronous data sources into a Compose State. It starts when entering Composition and cancels when leaving.
- **Conflation:** Setting the same value repeatedly does not trigger unnecessary recompositions.
- **Lowercase naming rule:** Compose convention for custom functions that return a State object (e.g., produceState, rememberState).

Question 11

- **Implementation:**

  Kotlin

```
@Composable
fun <T> myProduceState(
    initialValue: T,
    key: Any?,
    producer: suspend MutableState<T>.() -> Unit
): State<T> {
    val state = remember { mutableStateOf(initialValue) }
    LaunchedEffect(key) {
        state.producer()
    }
    return state
}
```

- **Behavior:** It behaves like LaunchedEffect because it wraps LaunchedEffect under the hood. When the key changes, the producer coroutine restarts.

Question 12

- **Why direct read causes recomposition:** Reading listState.firstVisibleItemIndex directly causes the composable to recompose on *every* scroll position change, even when the threshold (> 0) doesn't change.
- **How derivedStateOf helps:** It filters out high-frequency input changes, recomposing *only* when the derived boolean result (true / false) actually changes.
- **Why wrapped in remember:** To persist the derived state object across recompositions.
- **Math breakdown ([0, 0, 1, 2, 3, 4, 5, 3, 1, 0]):**

    - Direct read: Recomposes on every index change = **9 recompositions**.
    - derivedStateOf (evaluates index > 0 → [false, false, true, true, true, true, true, true, false): Changes occur at index 2 (false → true) and index 9 (true → false), plus initial composition = **3 recompositions**.

Question 13

- **Why pure overhead:** String interpolation "$firstName $lastName" changes every time either input changes; there is no reduction in frequency, making derivedStateOf redundant.
- **Correct version:**

  Kotlin

  val fullName = "$firstName $lastName"

- **Criterion for derivedStateOf:** Use it only when inputs change *more frequently* than the derived output.
- **Lecture Caution:** Do not overuse it for simple expressions where object allocation overhead outweighs performance savings.

Question 14

- **When to use derivedStateOf:** When high-frequency UI states (like scroll offsets or touch coordinates) map to infrequent UI triggers.
- **Three cases NOT to use:**
    1. *Precomputed amplitudes:* Already static or pre-processed, so no dynamic derivation is needed.
    2. *Static waveform:* Values never change, making derivation pointless.
    3. *Manual updates with LaunchedEffect or animate*:* Already managed or animated directly without high-frequency input volatility.

Question 15

- **snapshotFlow:** Converts Compose State reads into a cold Flow.
- **Operators:**
    - map: Transforms the index into a boolean (index > 0).
    - distinctUntilChanged: Emits only when the boolean value changes.
    - filter: Keeps only true values.
- **Why collect in LaunchedEffect:** To run the Flow collector safely tied to the Composable's composition lifecycle.
- **Model Stream (0, 0, 1, 2, 5, 3, 0, 0, 1, 4):**
    - Boolean values (index > 0): [false, false, true, true, true, true, false, false, true, true]
    - After distinctUntilChanged: [false, true, false, true]
    - After filter { it == true }: Emits **2 events** (the two transitions to true).

Question 16

Keys	Restarts (prediction)	Why
keys = []	0 times	Wrong: never restarts, uses stale values.
keys = [owner]	Restarts on owner change	Right if owner scopes it, but may miss tick updates.
keys = [owner, tick]	Restarts on every owner or tick change	Inefficient if tick updates shouldn't restart the effect.

Question 17

- **Keys vs Non-Keys:** lifecycleOwner must be an effect key. currentOnStart and currentOnStop must **not** be keys.
- **Why callbacks are not keys:** They are kept up-to-date using rememberUpdatedState without needing to restart the subscription.
- **Problem if lifecycleOwner is not a key:** If it changes, the observer remains attached to the old lifecycle owner, causing a memory leak or bug.
- **Key Point Rule:** Key effects by inputs that require restarting the subscription/coroutine; use rememberUpdatedState for changing callbacks.

Question 18

- **The Four Bugs & Fixes:**
  1. viewModel.markOpened(userId) in body → Runs on every recomposition. *Fix:* Move inside LaunchedEffect(userId).
  2. LaunchedEffect(true) → Ignores changes to userId. *Fix:* Change to LaunchedEffect(userId).
  3. DisposableEffect(owner) missing onDispose → Compilation error/leak. *Fix:* Add onDispose { owner.lifecycle.removeObserver(observer) }.
  4. LaunchedEffect(Unit) inside onClick callback → Invalid placement. *Fix:* Use scope.launch { send() }.

Question 19

API	Runs	Cleanup	Restarts on
LaunchedEffect	Enters Composition	Scope cancellation	Key changes

rememberCoroutineScope	On demand	Scope cancellation	N/A
rememberUpdatedState	Every recomposition	None	None
DisposableEffect	Enters Composition	onDispose block	Key changes
SideEffect	After every successful commit	None	None
produceState	Enters Composition	Scope cancellation	Key changes
derivedStateOf	When read & inputs change	None	Input state changes
snapshotFlow	When collected	Flow cancellation	State changes